W stronę sztucznej intelig...

Historia dostępna tylko dla członków

Sztuczna inteligencja | Narzędzia generatywnej sztucznej inteligencji | Odzyskiwanie rozszerzonej generacji | Okf | Agent AI

RAG kontra OKF: Przestań wybierać jedno. Dlaczego agenci AI potrzebują obu

RAG odzyskuje. OKF pamięta. Oto jak architektura się ze sobą łączy.



Rohan Mistry Podążać 9 minut czytania · 6 października 2026 r.

406 6 2

Co kilka miesięcy społeczność zajmująca się sztuczną inteligencją odkrywa nowego ***zabójcę RAG*** *.*

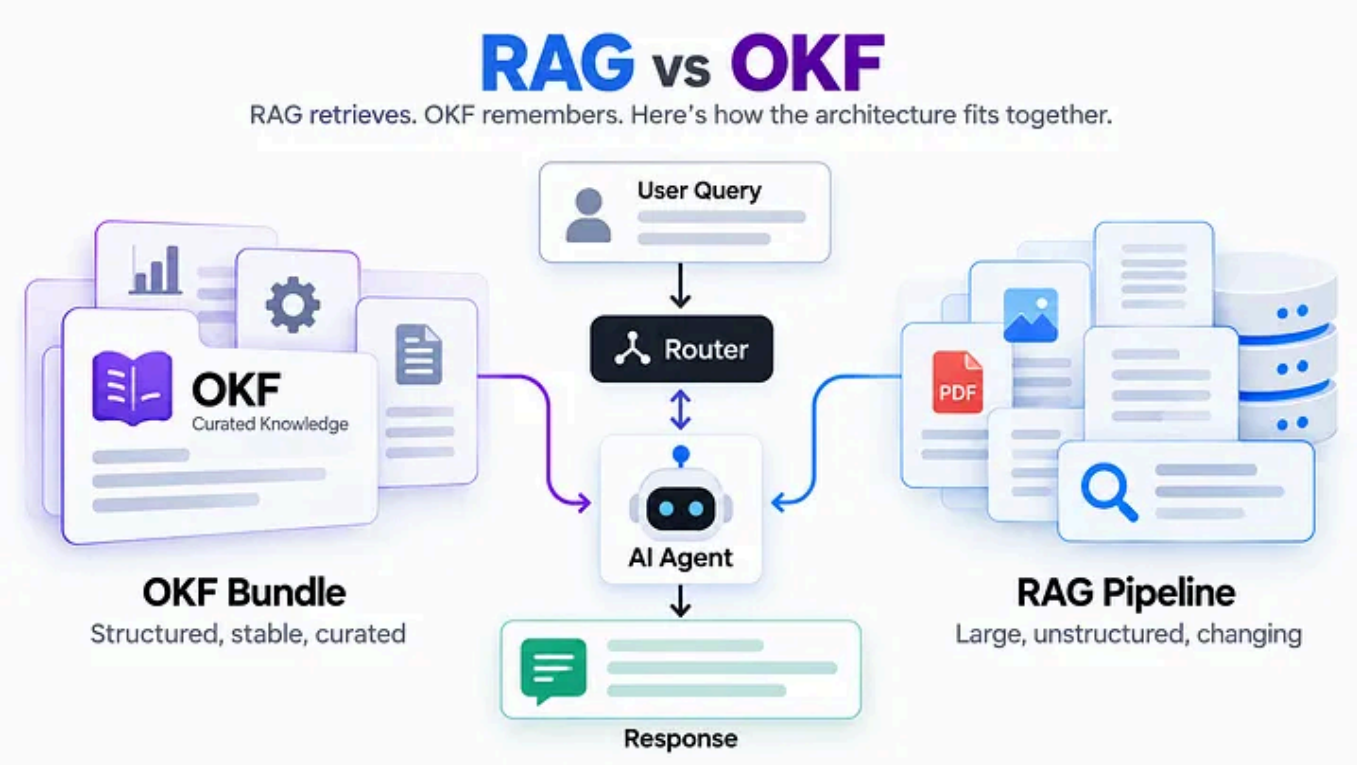
Dłuższe okna kontekstowe. Dostrajanie. GraphRAG. Agentic RAG.

Jednak w czerwcu 2026 r. Google Cloud opublikowało coś, co faktycznie zmieniło całą dyskusję: **Open Knowledge Format (OKF)** , otwartą, niezależną od dostawcy specyfikację umożliwiającą przechowywanie wyselekcjonowanej wiedzy w postaci zwykłych plików Markdown, które może bezpośrednio odczytać każdy agent AI.

> ***OKF nie zabija RAG. Ujawnia dokładnie to, czego RAG nigdy nie miał robić sam.***

W tym artykule szczegółowo opisano oba podejścia: *na czym polegają, jak działają, kiedy każde z nich jest skuteczne* i dlaczego najsilniejsze systemy AI w 2026 r. stosują oba podejścia łącznie.

Link dla osób niebędących członkami!



Jak czytać ten artykuł

RAG i OKF nie rywalizują o jedno miejsce. Rozwiązują różne problemy wiedzy:

- **RAG** odpowiada: „Informacje znajdują się gdzieś w tym dużym zbiorze, poszukaj ich”.
- **OKF** odpowiada: „Oto co już wiemy, wyrażone jasno, proszę to przeczytać”.

Mając to na uwadze, cała dyskusja traci sens.

1. RAG: Koń roboczy do odzyskiwania

Czym to właściwie jest

Model RAG (Retrieval-Augmented Generation) dodaje krok pobierania, zanim LLM wygeneruje odpowiedź. Zamiast polegać wyłącznie na danych treningowych, model przeszukuje zewnętrzną bazę wiedzy, pobiera odpowiednie fragmenty i wykorzystuje je jako kontekst do ugruntowania odpowiedzi.

```
Zapytanie → Osadzanie → Wyszukiwanie wektorowe → Fragmenty Top-K → LLM → Odpowie
```

RAG jest architekturą, do której sięgają przedsiębiorstwa w dziedzinie sztucznej inteligencji od 2023 roku. Rozwiązuje ona trzy kluczowe problemy: halucynacje (poprzez ugruntowanie odpowiedzi w rzeczywistych dokumentach), nieaktualność wiedzy (poprzez pobieranie aktualnych

informacji) oraz dostęp do prywatnych danych (poprzez indeksowanie wewnętrznych korpusów).

Uczciwy koszt, o którym nikt nie wspomina

RAG jest mocny, ale w jego prezentacji nie ma rachunku:

Podział na fragmenty to obszar, w którym większość systemów RAG po cichu zawodzi. Zbyt mały podział powoduje brak kontekstu. Zbyt duży podział powoduje rozmycie trafności szumem. Podział na fragmenty o stałym rozmiarze powoduje rozbicie jednostek semantycznych. Podział na fragmenty semantyczne zwiększa narzut inżynieryjny, na który nikt nie przeznacza budżetu na początku.

Jakość wyszukiwania to pułap. Jeśli wyszukiwarka pobierze niewłaściwe fragmenty lub całkowicie pominie właściwe, żadna szybka inżynieria nie uratuje wyniku. Naiwne potoki RAG zawodzą podczas wyszukiwania w około 40% przypadków.

Relacje są wnioskowane, nigdy jawne. RAG znajduje fragmenty, które są semantycznie *podobne* do zapytania. Ale podobieństwo to nie zrozumienie. Nie wie, że Koncepcja A *zależy od* Konceptu B ani że Polityka X *zastępuje*

Politykę Y. Te relacje zostają utracone, gdy dokumenty są dzielone na fragmenty po 512 tokenów.

Każde pytanie zaczyna się od zera. Nie ma kumulacji. LLM odkrywa na nowo wiedzę na każde pytanie. Nic się nie kumuluje. Nic nie kumuluje.

2. OKF: Warstwa wiedzy kuratorowanej, której RAG nigdy nie miał

Czym to właściwie jest

Open Knowledge Format (OKF), opublikowany przez Google Cloud 12 czerwca 2026 r. (wersja 0.1), to specyfikacja reprezentacji wiedzy organizacyjnej jako katalogu plików Markdown z frontmatterem YAML. Każdy plik reprezentuje jeden **koncept** : definicję metryki, schemat tabeli, politykę i podręcznik. Połączenia Markdown między plikami tworzą graf wiedzy, po którym agent może się poruszać.

```
knowledge-bundle/
  index.md # Wykaz katalogów (punkt wejścia)
  log.md #
```

```
Metryki historii zmian/
  index.md
  income.md # Jedna koncepcja na plik
  churn-rate.md
policies/
  escalation-matrix.md
  data-retention.md
```

Każdy plik koncepcyjny wygląda następująco:

```
---
typ:  metryka tytuł
:  Miesięczny  przychód cykliczny  właściciel: zespół finansowy tagi: [ przychód
```

Specyfikacja wymaga tylko jednego pola frontmatter: `type` . Cała reszta jest opcjonalna. Brak SDK. Brak zastrzeżonego środowiska uruchomieniowego. Brak wektorowej bazy danych. Brak osadzeń. Jeśli potrafisz czytać Markdown, potrafisz czytać OKF.

Historia powstania: Karpathy's LLM Wiki

OKF nie pojawił się znikąd. Formalizuje on wzorzec, który Andrej Karpathy opisał w X w kwietniu 2026 roku, na **LLM Wiki** . Jego spostrzeżenie było proste, ale radykalne: zamiast pobierać dane z surowych dokumentów w momencie zapytania, LLM powinien *skompilować* wiedzę w trwałą, powiązaną wiki i aktualizować ją.

Koncepcja Karpathy’ego trafnie ujmuje podstawowe ograniczenie RAG:

> ***Studia LLM pozwalają na ponowne odkrywanie wiedzy od podstaw w każdym pytaniu. Nie ma tu miejsca na kumulację.***

Jego post stał się viralem, notując ponad 21 milionów wyświetleń. Google Cloud przejął ten wzorzec i ujednolicił go w OKF dwa miesiące później.

Jak agent AI korzysta z OKF

W przeciwieństwie do potoku RAG, agent **nie** rozpoczyna pracy od przeszukiwania indeksu osadzania. Zaczyna od wybranego punktu wejścia i stopniowo eksploruje tylko koncepcje istotne dla bieżącego zadania:

```
1. Agent odczytuje plik index.md → sprawdza, jaka wiedza istnieje.
2. Agent podąża za linkami do odpowiednich plików konceptualnych
. 3. Agent odczytuje koncept: pełną, autorską, poprawną odpowiedź.
4. W razie potrzeby agent podąża za linkami do powiązanych konceptów.
5. Gotowe. Bez osadzania. Bez rankingu. Brak „najbliższego dopasowania”.
```

To jest **deterministyczne pobieranie** : albo znajdziesz plik w tej ścieżce, albo otrzymasz czysty błąd. Brak listy pięciu dokumentów, które *mogłyby* być istotne. Brak prawdopodobieństwa. Brak ukrytych błędnych odpowiedzi.

3. RAG kontra OKF: bezpośrednie starcie

Factor	RAG	OKF
Source shape	Large, changing, unstructured corpora	Curated, structured, stable knowledge
How it finds answers	Nearest-neighbor similarity search	Direct path lookup + explicit link traversal
Confidence in result	Probabilistic, "closest match"	Deterministic: the file exists or it doesn't
Relationships	Inferred from embeddings	Explicitly authored as markdown links
Updating a fact	Re-chunk, re-embed; old vectors can linger	Edit one file, commit, done
Auditability	Weak, hard to explain why a chunk was retrieved	Strong, with git history, file ownership, and explicit provenance
Token efficiency	Retrieves chunks + re-ranks + injects context	Agent reads the concept as-is, ~70× more token-efficient for curated facts
Scalability	Handles millions of documents	Requires manual curation; practical for hundreds to low thousands
Setup complexity	Embedding model + vector DB + chunking pipeline + reranker	A folder of markdown files
Best for	Large, messy, fast-changing document collections	Stable, authoritative knowledge that must be exactly right

4. Gdzie RAG nadal wygrywa (i zawsze będzie wygrywać)

OKF nie zastępuje RAG. Nigdy nie został zaprojektowany w tym celu. Oto, w jakich obszarach RAG pozostaje właściwym wyborem:

Skalowalność. Twoja organizacja ma 50 000 stron Confluence, dekadę zgłoszeń pomocy technicznej, tysiące wątków na Slacku i stale rosnącą stertę plików PDF. Zorganizowanie tego wszystkiego w starannie wyselekcjonowanych plikach koncepcyjnych OKF wymagałoby ogromnego nakładu pracy. RAG radzi sobie z tym bezproblemowo: osadza, indeksuje i przeszukuje.

Szybko zmieniające się dane. Ceny akcji, raporty o incydentach, logi w czasie rzeczywistym i interakcje z klientami zmieniają się nieustannie, dlatego lepiej nadają się do wyszukiwania semantycznego niż pliki Markdown.

Nieznane-nieznane. Kiedy nie wiesz, czego szukasz, potrzebujesz wyszukiwarki. RAG pozwala powiedzieć „znajdź mi wszystko, co jest związane z X” w obrębie ogromnego korpusu. OKF wymaga, aby ktoś już zapisał odpowiedź.

Odkrywanie. RAG doskonale radzi sobie z odkrywaniem powiązań, o których istnieniu nie wiedziałeś. Wyszukiwanie z osadzeniem może zwrócić istotny dokument sprzed trzech lat, którego nikt nie pamięta, że napisał.

5. Gdzie OKF wygrywa (a RAG zawodzi)

OKF to właściwy wybór, gdy uzyskanie dokładnej odpowiedzi jest ważniejsze niż przeszukiwanie dużego korpusu:

Zasady i podręczniki. Twoja macierz eskalacji ma jedną poprawną wersję. Agent, który ją śledzi `[[ops/policies/escalation_matrix]]`, albo znajdzie ten plik, albo zgłosi błąd. RAG może pobrać podobny, ale nieaktualny fragment z przestarzałej wersji.

Definicje metryk. Na pytanie „Co to jest MRR?” w Twojej organizacji istnieje jedna odpowiedź. Nie powinno to być wyszukiwanie podobieństw. Powinno to być bezpośrednie odczytanie.

Dokumentacja i schematy API. Schematy tabel, specyfikacje punktów końcowych i słowniki danych są dokładne, ustrukturyzowane i wiarygodne. Należy je umieszczać w uporządkowanych plikach koncepcyjnych, a nie w pogrupowanych indeksach wektorowych.

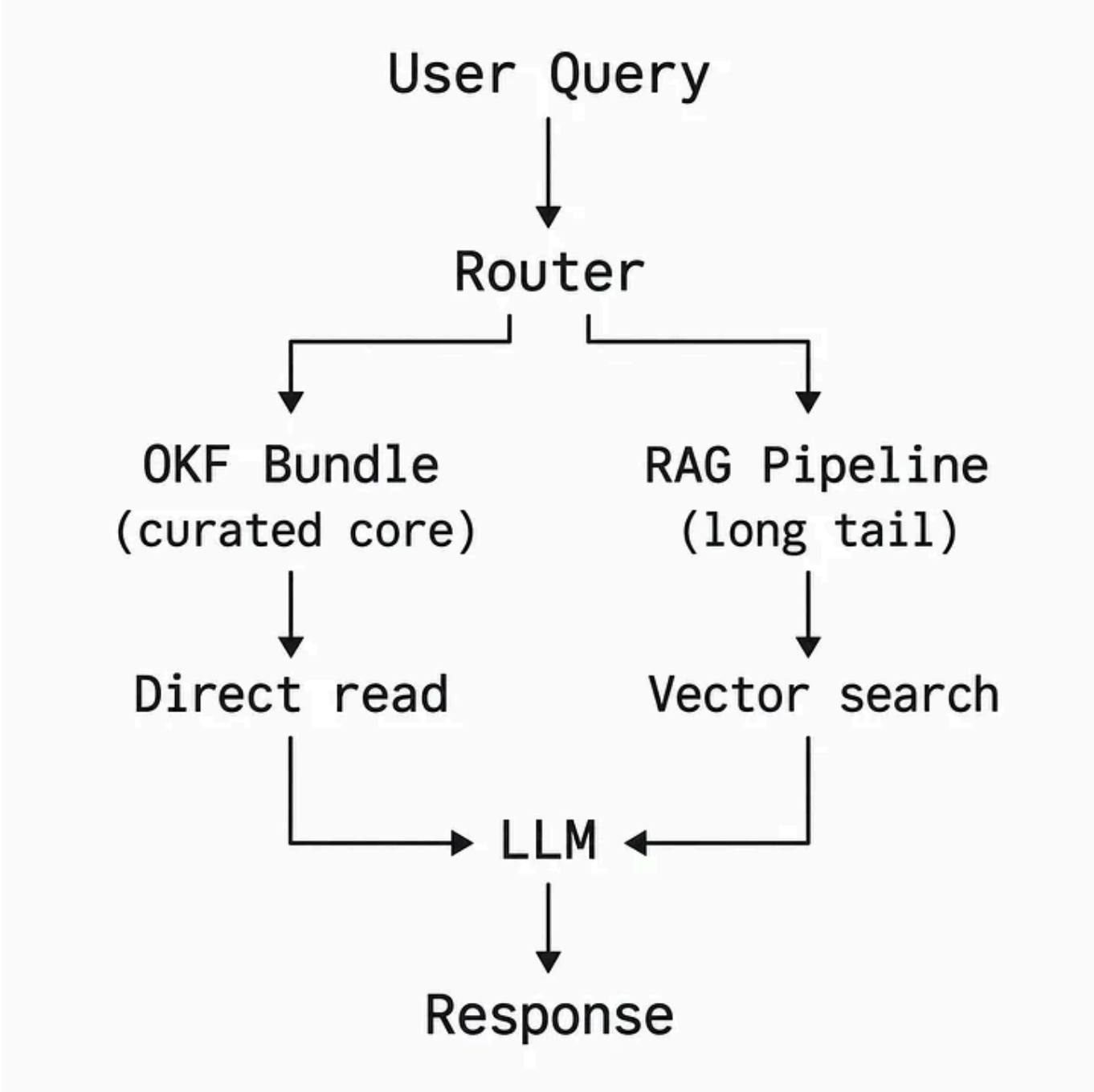
Wdrażanie i szkolenia. Wdrażanie nowych inżynierów, materiały szkoleniowe dotyczące zgodności oraz procedury operacyjne (SOP) to wiedza stabilna, która zmienia się rzadko i musi być precyzyjnie poprawna.

Zgodność i audyt. Branże regulowane (bankowość, ubezpieczenia, opieka zdrowotna) muszą odpowiedzieć na pytanie: „Co ta polityka mówiła tego dnia?”. Pakiety OKF są dostępne w Git. Każda zmiana jest śledzona. Każdy koncept ma właściciela. Każda relacja jest jawna. Spróbuj przeprowadzić audyt bazy danych wektorowych.

6. Architektura, która naprawdę wygrywa: OKF + RAG

A oto puenta, której brakuje w debacie na temat tego, „co wygrywa”: **systemy produkcyjne wykorzystują oba .**

Architektura, która będzie konwergować między zespołami w 2026 r., to **router** znajdujący się przed obydwoma systemami:


User Query
Router
OKF Bundle
(curated core)
RAG Pipeline
(long tail)
Direct read
Vector search
LLM
Response

Podział 80/20

Praktyczny wzorzec pojawiający się w różnych przedsiębiorstwach:

- **20%** wiedzy organizacyjnej, która jest krytyczna, stabilna i często przywoływana, istnieje w postaci **pakietu OKF** : zasad, metryk, schematów, podręczników i procedur operacyjnych SOP.
- **Pozostałe 80%** , czyli obszerne, niestrukturyzowane i szybko zmieniające się informacje, pozostaje w tradycyjnym **procesie RAG** .

Router działa prosto: znane-znane trafiają do OKF. Wszystko inne trafia do RAG.

OKF ulepsza RAG

Oto część, której brakuje w większości artykułów: pakiet OKF to czysty, dobrze ustrukturyzowany kod Markdown, który jest bliski *idealnemu wejściu do potoku fragmentacji RAG* . Skieruj swój program do pobierania na pakiet OKF, a otrzymasz lepsze fragmenty niż kiedykolwiek dałoby surowe dokumenty źródłowe.

OKF poszerza wiedzę. RAG nadal dba o to, aby była ona dostępna na dużą skalę.

7. Pierwsze kroki: Twój pierwszy pakiet OKF w 5 minut

Nie potrzebujesz nowego narzędzia. Potrzebujesz folderu i edytora tekstu.

Krok 1: Utwórz strukturę pakietu

```
mkdir -p pakiet wiedzy/metryki pakiet wiedzy/polityki
dotyk pakiet wiedzy/index.md pakiet wiedzy/log.md
```

Krok 2: Napisz swój pierwszy pomysł

Tworzyć `knowledge-bundle/metrics/mrr.md`:

```
---
typ:  metryka
tytuł:  Miesięczny  przychód cykliczny  właściciel: finance-team tagi: [ przychó
```

Krok 3: Zbuduj indeks

W `knowledge-bundle/index.md`:

```
# Pakiet wiedzy

## Metryki
- [ Miesięczny przychód cykliczny ]( ./metrics/mrr.md ): Podstawowa metryka przy

## Zasady
- [ Macierz eskalacji ]( ./policies/escalation-matrix.md ): Trasowanie reakcji n
```

Krok 4: Wskaż to swojemu agentowi

Medium Szukaj Graj w gry Pisać

pojęć i ma deterministyczny dostęp do wyselekcjonowanej, wiarygodnej wiedzy, bez konieczności osadzania treści.

Krok 5: Utwórz wersję

```
cd knowledge-bundle && git init && git add . && git commit -m "Początkowy pakiet
```

Twoja wiedza jest teraz wersjonowana, porównywalna, możliwa do zweryfikowania i przenośna.

8. Ekosystem OKF (październik 2026)

OKF ma już cztery miesiące i już dysponuje rozwijającym się zestawem narzędzi:

- **okflib (Python)** : analizowanie, weryfikacja, wizualizacja i automatyczne generowanie pakietów OKF
- **okf (Rust)** : parser o zerowej zależności, walidator i graf połączeń
- **okf-toolkit (Python CLI)** : polecenia init, validate, search i graph

- **okf (Go)** : Pełny model pakietu v0.2 ze sprawdzaniem zgodności
- **Hermes OKF** : Uniwersalny system pamięci dla agentów AI zbudowany na OKF
- **OKF Wiki** : Baza wiedzy wspierana przez Obsidian i utrzymywana przez LLM, oparta na OKF
- **Serwery MCP** : natywne narzędzia OKF dla Claude, Codex i innych agentów

Specyfikacja jest licencjonowana przez Apache 2.0. Brak uzależnienia od dostawcy. Brak zastrzeżonego środowiska uruchomieniowego.

9. Szybkie ramy decyzyjne

Scenario	Use
< 100 structured, stable documents	**OKF only**
100–1,000 with structure	**OKF first, add RAG at scale**
1,000+ documents, mixed structure	**RAG required (OKF for the curated core)**
Content rarely changes	**OKF**
Content changes frequently	**RAG (or hybrid with re-ingestion automation)**
Tables, SOPs, specs, FAQs	**OKF**
Support tickets, Slack, meeting notes	**RAG**
Exact answers required	**OKF**
Discovery and exploration	**RAG**
Compliance and auditing needs	**OKF**
Massive unstructured corpus	**RAG**
Enterprise agent that needs both	**Hybrid: route known-knowns to OKF, long tail to RAG**

10. Typowe błędy

„OKF zastępuje RAG”. Nie. OKF zastępuje wyszukiwanie wektorowe tylko w przypadku wybranych faktów, do których można się bezpośrednio odwołać. Długi ogon pozostaje dokładnie tam, gdzie był.

„RAG wystarczy do wszystkiego”. Nie wystarczy. Gdy agent musi mieć *pewność* co do polityki, metryki lub schematu, wyszukiwanie probabilistyczne nie jest odpowiednim narzędziem.

„OKF do wszystkiego”. Przekształcanie tysięcy szybko zmieniających się dokumentów w koncepcje OKF jest niepraktyczne. Użyj OKF do kluczowego rdzenia, a RAG do reszty.

„Muszę pozbyć się mojego systemu RAG”. Nie musisz. Pierwszym praktycznym krokiem jest wybranie pięciu do dziesięciu odpowiedzi, na które Twój agent najczęściej udziela błędnych odpowiedzi – tych z jedną poprawną odpowiedzią – i zapisanie ich najpierw jako konceptów OKF.

„OKF potrzebuje specjalnego narzędzia”. Potrzebuje edytora tekstu i folderu. O to właśnie chodzi.

Czego tak naprawdę oczekują ankieterzy w 2026 roku

Na stanowiskach związanych ze sztuczną inteligencją/uczeniem maszynowym i inżynierią danych należy być gotowym na:

- **Wyjaśnij, dlaczego „RAG kontra OKF" to błędne ujęcie.** Rozwiązują one różne problemy wiedzy: znane-znane kontra nieznane-nieznane.
- **Opisz ograniczenia strukturalne RAG.** Kruchość fragmentacji, wyszukiwanie probabilistyczne, wnioskowane zależności, brak akumulacji.
- **Wyjaśnij, czym jest OKF.** Pliki Markdown + materiały wstępne YAML + jawne linki = deterministyczna, uporządkowana wiedza dla agentów.
- **Poznaj powiązania z Karpathy.** OKF formalizuje wzorzec LLM Wiki (kwiecień 2026 r.) w przenośny, niezależny od dostawców standard.
- **Naszkicuj architekturę hybrydową.** Router → OKF dla rdzenia, RAG dla długiego ogona.
- **Jasno określ kompromisy.** OKF zapewnia strukturę i precyzję. RAG zapewnia skalę i elastyczność. Systemy produkcyjne wykorzystują oba.

Obejmując te sześć punktów, będziesz rozumować o systemach wiedzy AI jak architekt, a nie fan.

Na wynos

RAG i OKF nigdy nie walczyły o ten sam tron.

RAG to prawdziwy „koń roboczy” w dziedzinie wyszukiwania: przeszukuje duże, chaotyczne, szybko zmieniające się korpusy i wyświetla istotne informacje w momencie zapytania.

OKF to warstwa skatalogowanej wiedzy: stabilna, autorytatywna, deterministyczna wiedza, którą agenci odczytują bezpośrednio, z wyraźnymi relacjami i pełną audytowalnością.

Błędem jest traktowanie ich jak rywali. To warstwy różnych fragmentów twojej wiedzy:

- Niewielkie **, krytyczne i stabilne 20%** znajduje się w OKF: politykach, metrykach, schematach i podręcznikach.

- Duże **, niestrukturyzowane, dynamiczne 80%** pozostaje w RAG: bilety, wątki, notatki ze spotkań i dokumenty historyczne.

Najsilniejsze systemy AI w 2026 roku to nie te, które spierają się o to, które podejście jest lepsze. To te, które używają routera, który umieszcza każde z nich tam, gdzie wygrywa: **OKF dla tego, co musi być idealnie poprawne, RAG dla wszystkiego, co jest zbyt duże, by tworzyć je ręcznie.**

Przestań pytać, co jest lepsze. Zapytaj, co musi być *pewne* dla Twojego agenta i zacznij od tego.

Jeśli ten artykuł okazał się przydatny, śledź nas, aby dowiedzieć się więcej na temat sztucznej inteligencji, uczenia maszynowego i systemów danych.

Sztuczna inteligencja + | Narzędzia generatywnej sztucznej inteligencji + | Odzyskiwanie rozszerzonej generacji + | Okf + | Agent AI +



Napisane przez Rohana Mistry'ego

1,8 tys. obserwujących · 42 następujące

Inżynier AI/ML skupiający się na budowaniu rzeczywistych inteligentnych systemów i skalowalnych rozwiązań AI.

Podążać

Kup mi kawę

Teczka

GitHub

AI Web Studio

Zobacz więcej ode mnie

Opublikowano w Towards AI

Podążać

150 tys. obserwujących · Ostatnio opublikowano przed chwilą

Szkolimy inżynierów AI. Wdrażamy AI w przedsiębiorstwach. Dołącz do ponad 100 tysięcy praktyków AI w Towards AI Academy lub skontaktuj się z nami, aby uzyskać pomoc w adaptacji AI w Twojej firmie. towardsai.com

Pomoc Status O Kariera Naciskać Blog Sklep Prywatność Zasady Warunki Tekst na mowę